

컴퓨터비전 개론 1주차 요약본(5. 13.~5. 15.)

1. 딥러닝 필수 파이썬, 실습 환경구성 및 OOP

1.1 개발 환경 설정

- **Anaconda란?**
 - 데이터 분석과 머신러닝을 위한 필수 라이브러리를 포함한 배포판
 - Pandas, NumPy, Matplotlib 등 주요 라이브러리 내장
 - Spyder, Jupyter Notebook 등 개발 도구(IDE) 제공
 - 버전 관리와 패키지 업데이트가 편리함
 - Windows, MacOS, Linux에서 모두 사용 가능
- **Anaconda 설치 과정**
 - [Anaconda 공식 홈페이지](#) 접속
 - 운영체제에 맞는 Anaconda 다운로드
 - 설치 후 환경 변수를 설정하여 conda 명령어를 사용할 수 있도록 함
- **Google Colab이란?**
 - Google에서 제공하는 무료 클라우드 기반 Jupyter Notebook 환경
 - 로컬 환경 설정 없이 머신러닝 및 딥러닝 개발 가능
 - GPU 및 TPU 지원
 - Google Drive와 연동하여 코드 및 데이터 저장 가능
- **Google Colab 접속 방법**
 - Google 계정으로 로그인
 - [Google Colab 접속](#) 후 "New Notebook" 선택
 - 런타임 유형을 GPU 또는 TPU로 변경 가능

1.2 딥러닝 개발 순서

- 문제 정의 – 해결할 목표 설정
- 데이터 수집 – 학습용 데이터 확보
- 전처리 – 데이터 정제 및 가공
- 모델 설계 – 모델 구조 및 설정 결정
- 학습 – 모델 훈련 및 파라미터 최적화
- 평가 – 성능 측정 및 검증
- 개선 – 하이퍼파라미터 조정 및 성능 향상
- 배포 – 운영 환경에 적용
- 모니터링 – 성능 추적 및 재학습 관리

1.3 Pytorch

- 페이스북이 개발한 딥러닝 프레임워크
- 직관적인 코드 작성 가능 → 연구 및 실무에서 많이 사용
- 동적 연산 그래프 기반으로 디버깅과 실험이 쉬움
- GPU 연산 지원으로 빠른 학습 가능

- 행렬의 구조

```
A = np.array([[1, 2, 3], [4, 5, 6]])  
print(A)
```

- 행렬의 연산

```
B = np.array([[7, 8, 9], [10, 11, 12]])
```

```
C = A + B # 행렬 덧셈
```

```
D = A - B # 행렬 뺄셈
```

-> 같은 크기의 행렬끼리만 덧셈/뺄셈 가능

```
E = np.dot(A, B.T) # A와 B의 전치 행렬을 곱함
```

-> 일반적인 행렬 곱셈을 수행할 때는 np.dot() 사용

-> 원소별 곱셈(Element-wise multiplication)은 A * B 사용

```
np.linalg.inv(A) # 역행렬
```

```
np.linalg.det(A) # 행렬식 계산
np.linalg.eig(A) # 고유값과 고유벡터 계산
-> 선형대수 연산 가능 (linalg 모듈 활용)
```

1.4 자동 미분과 수치 미분

구분	자동 미분 (Autograd)	수치 미분 (Numerical)
방식	연산 그래프 기반 기울기 계산	근사값 계산
정확도	매우 높음	h값에 따라 오차 존재
속도	빠름	느림
활용	학습 과정, 딥러닝 모델	검증, 디버깅

1.5 파이썬 객체 지향

- 클래스 (Class)
 - 객체를 만들기 위한 설계도
 - 속성과 메서드를 정의함.
- 인스턴스 (Instance)
 - 클래스로부터 생성된 실제 객체
- 속성 (Attribute)
 - 객체가 가지는 데이터(상태)
- 메서드 (Method)
 - 클래스 내부에 정의된 함수로, 객체의 동작을 정의

```
class Dog:
    def __init__(self, name):
        self.name = name # 속성
    def bark(self):      # 메서드
        print("멍멍!")

dog1 = Dog("바둑이") # Dog 클래스의 인스턴스
print(dog1.name)    # '바둑이'
dog1.bark()         # "멍멍!"
```

2. Numpy, Matplotlib, 행렬 연산 기초

2.1 Numpy(Numerical Python)

- 행렬의 구조 및 연산
- 인덱싱 및 슬라이싱
- 행렬의 덧셈, 뺄셈, 곱셈, 전치(Transpose)

- **Numpy 기본 구조**

- 행렬의 구조

```
A = np.array([[1, 2, 3], [4, 5, 6]])  
print(A)
```

- 행렬의 연산

```
B = np.array([[7, 8, 9], [10, 11, 12]])
```

```
C = A + B # 행렬 덧셈
```

```
D = A - B # 행렬 뺄셈
```

-> 같은 크기의 행렬끼리만 덧셈/뺄셈 가능

```
E = np.dot(A, B.T) # A와 B의 전치 행렬을 곱함
```

-> 일반적인 행렬 곱셈을 수행할 때는 np.dot() 사용

-> 원소별 곱셈(Element-wise multiplication)은 A * B 사용

```
np.linalg.inv(A) # 역행렬
```

```
np.linalg.det(A) # 행렬식 계산
```

```
np.linalg.eig(A) # 고유값과 고유벡터 계산
```

-> 선형대수 연산 가능 (linalg 모듈 활용)

2.2 Numpy 속성 및 연산

- **Numpy 배열 속성**

```
array = np.array([[1, 2, 3],  
                  [4, 5, 6],  
                  [7, 8, 9]], dtype=np.int32)
```

```

print(array)
print("array data type =", array.dtype) # 데이터 타입
print(array.shape) # 배열의 형태 (행, 열)
print(array.ndim) # 차원 개수
print(array.size) # 전체 요소 개수

```

• 특별한 행렬 생성

```

print(np.eye(2, 3)) # 단위 행렬 (대각선만 1, 나머지는 0)
print(np.identity(3)) # 정방형 단위 행렬
print(np.zeros([3,3])) # 0으로 채워진 행렬
print(np.ones((3,5))) # 1로 채워진 행렬
print(np.full((3,5), 5)) # 5로 채워진 행렬

```

단위 행렬 (eye, identity)
 0으로 채운 행렬 (zeros)
 1로 채운 행렬 (ones)
 특정 값으로 채운 행렬 (full)

• Numpy 연산

```

x = np.arange(12) # 0~11까지 연속된 숫자 생성
y = np.linspace(0, 11, 12) # 0~11까지 12개 균등 분포 값 생성
arange(start, stop, step) → 정수 시퀀스 배열
linspace(start, stop, num) → 균등 간격으로 값 생성

```

```

x = np.arange(1, 7).reshape(2, 3)
print(x.T) # 전치행렬 (행과 열 교환)

```

```

np.matmul(x, x.T)
-> 행렬 곱 연산 (일반적인 곱셈)

```

```

x = np.array([[[0], [1], [2]]])
print(x.squeeze()) # 불필요한 차원 제거, 배열 차원 축소
.squeeze() → 차원이 1인 축을 제거

```

```

list1 = [[1, 2, 3]]
list2 = [[4, 5, 6]]
print(np.concatenate([list1, list2], axis=0)) # 행 방향 연결
print(np.concatenate([list1, list2], axis=1)) # 열 방향 연결
print(np.vstack((list1, list2))) # 행 방향 연결
print(np.hstack((list1, list2))) # 열 방향 연결

a = np.arange(4).reshape((2,2))
print(np.min(a)) # 전체 배열의 최소값
print(np.min(a, axis=0)) # 열 방향 최소값
print(np.max(a, axis=1)) # 행 방향 최대값

.min()/ .max() → 최소값/최대값 찾기 (axis=0: 열 기준, axis=1: 행 기준)

```

• 난수 생성

```

# 균등 분포 난수 (rand)
np.random.rand(3,2)

rand() → [0,1] 사이에서 랜덤 샘플링

# 정수 난수 (randint)
np.random.randint(1, 10, (3,2)) # 1~10 사이의 정수 난수

randint(low, high, size) → 정수 난수 생성

# 정규분포 난수 (randn)
np.random.seed(123) # 고정된 결과를 얻기 위한 시드 설정
np.random.randn(3,2) # 평균 0, 표준편차 1의 정규 분포 난수
randn() → 표준 정규 분포 (mean=0, std=1)에서 난수 샘플링
seed(n) → 랜덤 시드 설정 (결과 고정 가능)

```

2.3 Matplotlib (데이터 시각화)

- Matplotlib 구조

```
import matplotlib.pyplot as plt

x = np.linspace(-10, 10, 100) # -10부터 10까지 100개 점 생성
y = np.sin(x)

plt.plot(x, y) # 선 그래프 그리기
plt.xlabel("X-axis") # x축 제목 설정
plt.ylabel("Y-axis") # y축 제목 설정
plt.title("Sine Wave") # 그래프 이름 설정
plt.grid() # 격자 표시
plt.show() # 그래프 출력
```

2.4 Matplotlib 활용

- 시그모이드 함수 그래프

```
# 시그모이드 함수 정의 및 그래프 그리기
def sigmoid(x, a):
    return 1/(1 + np.exp(-a*x))

xp = np.linspace(-3, 3, 61)
yp = sigmoid(xp, 1.0)
yp2 = sigmoid(xp, 2.0)

plt.plot(xp, yp)
plt.show()

시그모이드 함수 정의:  $\text{sigmoid}(x) = \frac{1}{1 + \exp(-a \cdot x)}$ 
xp → x값 (-3 ~ 3 범위)
yp, yp2 → 시그모이드 함수 결과값
```

- Subplot(다수의 그래프 동시 출력)

```
# MNIST 데이터셋 시각화
from sklearn.datasets import fetch_openml
```

```

mnist = fetch_openml('mnist_784', version=1, as_frame=False)

image = mnist.data # 이미지 데이터
label = mnist.target # 정답 데이터

plt.figure(figsize=(10, 3))

for i in range(20):
    ax = plt.subplot(2, 10, i+1)
    img = image[i].reshape(28,28) # 28x28 픽셀로 변환
    ax.imshow(img, cmap='gray_r') # 흑백으로 출력
    ax.set_title(label[i]) # 타이틀(정답) 추가
    ax.set_xticks([]) # x축 눈금 제거
    ax.set_yticks([]) # y축 눈금 제거

plt.tight_layout() # 그래프 간격 조정
plt.show()

```

3. Pytorch 기본: 텐서(Tensor) 핵심

3.1 PyTorch 개요

- **PyTorch란?**
 - 딥러닝 신경망 구축을 위한 오픈소스 프레임워크
 - Torch 머신러닝 라이브러리 + Python API를 결합
 - 쉽고 직관적인 문법을 제공하며, 동적 연산이 가능

- **PyTorch&TensorFlow&Keras 비교**

항목	Pytorch	Tensorflow	Keras
개발사	Meta(Pytorch Foundation)	Google	Google (TensorFlow 상위 인터페이스)
사용 분야	연구, 학계 등	산업용, 배품 제포 등	교육, 빠른 프로토타이핑

항목	Pytorch	Tensorflow	Keras
장점	사전 학습 모델과 문서가 많음	Google 서비스와 통합이 용이함	사용 간편

• Pytorch 기본 구조 자료형

◦ 텐서 (Tensor)

- PyTorch의 핵심 데이터 구조
- NumPy 배열과 유사하지만 GPU 가속이 가능
- 스칼라 (Scalar) — 0차원 텐서
- 벡터 (Vector) — 1차원 텐서
- 행렬 (Matrix) — 2차원 텐서
- 텐서 (Tensor) — 3차원 이상

```
import torch
# 스칼라 (0차원 텐서)
a = torch.tensor(5)
print(a.shape)
# 벡터 (1차원 텐서)
v = torch.tensor([1, 2, 3])
print(v.shape)
# 행렬 (2차원 텐서)
m = torch.tensor([[1, 2], [3, 4]])
print(m.shape)
# 3차원 텐서 (예: 배치 x 행 x 열)
t = torch.randn(2, 3, 4)
print(t.shape)
```

◦ PyTorch 자료형

- torch.FloatTensor → 32비트 실수형 텐서
- torch.IntTensor → 32비트 정수형 텐서
- torch.cuda.FloatTensor → GPU 가속된 실수형 텐서

3.2 PyTorch를 활용한 미분 및 경사 계산

```
# Pytorch 텐서를 Numpy로 변환
r2_np = r2.data.numpy()
print(type(r2_np)) # NumPy 배열 타입 확인
print(r2_np) # 변환된 값 출력
PyTorch 텐서를 NumPy 배열로 변환 (.data.numpy())
CPU 상에서 NumPy 연산과 PyTorch 연산을 혼합하여 사용할 수 있음

#2차 함수의 경사(미분) 계산
x_np = np.arange(-2, 2.1, 0.25) # -2부터 2까지 0.25 간격의 NumPy 배열 생성
x = torch.tensor(x_np, requires_grad=True, dtype=torch.float32)
-> requires_grad=True
    → PyTorch가 자동으로 미분(Gradient)을 계산할 수 있도록 설정

# 2차 함수  $y = 2x^2 + 2$ 을 정의
y = 2 * x**2 + 2
print(y)

# 계산 그래프 자동 생성 및 시각화
z = y.sum() # 경사 계산을 위해 sum() 사용
from torchviz import make_dot
g = make_dot(z, params={'x': x}) # 계산 그래프 시각화
display(g)
PyTorch 내부에서 자동으로 미분 연산을 위한 계산 그래프 생성
torchviz를 활용하여 그래프를 시각화
```

3.3 자동 미분(Autograd)으로 경사 계산

```
# 경사(Gradient) 계산
z.backward() # 자동 미분 수행
print(x.grad) # x에 대한 미분값 출력

backward()를 호출하면 requires_grad=True가 설정된 텐서의 미분값이 자동으로 계산
출력된 값은 2차 함수  $y = 2x^2 + 2$ 의 도함수인  $dy/dx = 4x$ 
```

```
# 함수(y)와 미분값(Gradient) 그래프를 동시에 출력하여 변화율 시각화
plt.plot(x.data, y.data, c='b', label='y') # 함수 그래프
plt.plot(x.data, x.grad.data, c='k', label='y.grad') # 경
사 그래프
plt.legend()
plt.show()

# 경사를 초기화하지 않으면 값이 누적
y = 2 * x**2 + 2
z = y.sum()
z.backward()
print(x.grad)
backward()를 두 번 실행하면 경사 값이 누적됨

# 경사 초기화 (zero_())
x.grad.zero_()
print(x.grad)
zero_()를 사용하여 경사값을 초기화
PyTorch는 기본적으로 경사값을 누적하므로, 매번 초기화하는 것이 중요
```

3.4 Pytorch 활용하여 시그모이드 함수와 자동 미분(Gradient Calculation)

- 시그모이드 함수 정의 및 계산

```
# PyTorch 내장 Sigmoid 함수 사용
sigmoid = torch.nn.Sigmoid()
y = sigmoid(x)
PyTorch의 torch.nn.Sigmoid()를 이용하여 시그모이드 함수 정의
입력 x에 대한 y = sigmoid(x) 값 계산

# 스칼라 값으로 변환
z = y.sum() # 자동 미분을 수행하려면 최종 값이 스칼라여야 함
PyTorch의 자동 미분(Autograd)을 사용하려면,
최종 결과를 하나의 스칼라 값으로 변환해야 함
여기서는 sum()을 사용하여 모든 값을 하나의 값으로 합산
```

```
# 계산 그래프 시각화
g = make_dot(z, params={'x': x})
display(g)
torchviz를 사용하여 계산 그래프 시각화
```

• 자동 미분(Gradient Calculation)

```
# 미분(경사) 계산
z.backward()
print(x.grad) # x에 대한 경사(미분값) 출력
backward()를 호출하면 PyTorch가 자동으로 미분을 계산
출력된 값은 dy/dx로, 시그모이드 함수의 도함수를 나타냄

# 경사값 초기화 (zero_())
x.grad.zero_()
print(x.grad)
PyTorch는 기본적으로 경사값을 누적하므로,
새로운 계산을 위해 zero_()를 사용하여 초기화해야 함
```

• 사용자 정의 시그모이드 함수 구현

```
# PyTorch 내장 함수 없이 직접 시그모이드 함수 구현
def sigmoid(x):
    return 1 / (1 + torch.exp(-x))

# 사용자 정의 함수에 대한 그래디언트 계산
y = sigmoid(x)
-> 정의한 sigmoid(x)를 사용하여 y값을 계산

z = y.sum() # 스칼라 값으로 변환
params = {'x': x}
g = make_dot(z, params=params)
display(g)

z.backward() # 자동 미분 수행
print(x.grad) # 경사 출력
```

